# 1. Abstract

Heart disease is one of the most dangerous health problem in the world today, and early detection can help to save many lives. The main objective of this project is to develop a Heart Disease Detection System using machine learning techniques. The system helps in predicting whether a patient have heart disease or not by analysing different medical features.In this project, the UCI Heart Disease dataset was used, which contains 920 patient records with 15 clinical attributes such as age, blood pressure, cholesterol level, chest pain type, and heart rate. Since the dataset contains some missing values and categorical data, preprocessing was done before training the models. Missing values was handled carefully and one-hot encoding technique was applied to convert categorical values into numerical format.Four machine learning algorithms was implemented and compared in this project: Logistic Regression, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. The dataset was divided into 80% training data and 20% testing data for evaluation purpose. After comparing the performance of all models, XGBoost gave the best result with an accuracy of 84.24%, F1-score of 0.8612, and AUC-ROC score of 0.8462.The trained model and scaler was saved using Pickle so that it can be used later for deployment and future applications. This project shows that machine learning can be very useful in helping doctors and healthcare professionals for early heart disease prediction and better decision making.

# 2. Table of Contents

# 3. Introduction

- Overview of machine learning in healthcare

  Machine learning is becoming very common in healthcare field because it helps doctors to understand patient data in faster and easier way. It can find some hidden patterns and health problems which sometimes are difficult to identify by normal methods. In heart disease diagnosis, early prediction is very important for patients health. Machine learning models also helps doctors in taking quick decisions and reduce some workload in hospitals and improve healthcare services also.

- Importance of disease prediction

  Machine learning is now becoming more popular in healthcare sector because it helps doctors to study patient information in quick and simple way. It can detect some important patterns and diseases which are not easy to find using traditional methods. In case of heart disease, early detection is very necessary for better treatment of patients. Machine learning models also help doctors in making faster decisions and it reduce workload of hospital staffs and improve healthcare system overall.

- Motivation for the project

  Even though large amount of clinical data is available in hospitals, many healthcare providers still do not have proper tools to analyse and understand it quickly. Because of this, diagnosis process can become slow sometimes. This project tries to solve that problem by developing a machine learning based system which can predict heart disease automatically. It helps in giving faster results and provide more objective support to doctors during clinical decision making.

# 4. Problem Statement

Heart disease diagnosis mostly depends on expensive medical tests, availability of specialist doctors, and clinical judgement which can sometimes vary from person to person. In many rural or resource limited areas, patients often face delay in proper diagnosis and treatment. This project tries to solve this problem by developing a machine learning based system which can predict whether a patient have heart disease or not using normal clinical parameters. It can help in early detection and provide low cost screening support for patients.

# 5. Objectives

• To build a binary classification model which can predict that a patient has heart disease or not.

• To compare different machine learning algorithms like Logistic Regression, Random Forest, KNN, and XGBoost for finding which one gives more better result.

• To handle some real world dataset problems such as missing values and mixed type data which are present in the dataset.

• To achieve good accuracy, precision, recall, and AUC-ROC score so the prediction performance can become more effective.

• To save the best trained model using Pickle serialization so it can be used again later for deployment and future applications purpose.

# 6. Literature Review

Briefly discuss:

**Existing Disease Prediction Systems**

Many research studies have already worked on machine learning based heart disease prediction systems. In earlier studies, researchers mostly used the Cleveland Heart Disease dataset from UCI repository along with algorithms like Naive Bayes and Decision Trees. These models was giving around 75–80% accuracy in most cases. In recent years, more advanced methods like ensemble learning and gradient boosting have been used by researchers, and the prediction accuracy have improved above 85% by using larger and more diverse datasets.ML Techniques Used in Healthcare

Logistic Regression is widely used for health classification because it is simple and easy to understand. Random Forest and XGBoost became more popular because they can handle different type of data more effectively. KNN also gives good result when data is properly scaled. Deep learning models like CNN and LSTM are also being explored, but they requires much larger datasets for better performance. Limitations of Existing Approaches

- Most studies use only the Cleveland subset; this project uses the full multi-source UCI dataset (920 records)
- High missing data rates (especially in 'ca' at 66.41% and 'thal' at 52.83%) are rarely addressed systematically
- Many studies skip model comparison and select a single algorithm without justification
- Deployment-readiness (model serialization, scaler saving) is often omitted

# 7. Methodology

### 7.1 Data Collection

The dataset used in this project is the UCI Heart Disease dataset taken from Kaggle (shantanusharma23410/heart-disease-csv). It includes patient records collected from four different medical institutes which are Cleveland, Hungary, Switzerland, and VA Long Beach. The original dataset contains around 920 rows and 16 columns, including the target variable called ‘num’ which is used for heart disease prediction

### 7.2 Data Preprocessing

Preprocessing was performed in the following sequence:

- Target column fix: The original 'num' column had values 0-4 indicating severity. It was binarized — 0 (no disease) and 1 (disease present for any value > 0)
- Column 'id' dropped: No predictive value, acts only as a row identifier
- Column 'dataset' dropped: Represents data source, not a clinical feature
- Missing value analysis: 'ca' had 66.41% missing, 'thal' had 52.83%, 'slope' had 33.59% — all imputed using median (numerical) and mode (categorical)
- One-hot encoding: All categorical columns (sex, cp, fbs, restecg, exang, slope, thal) were expanded into binary dummy variables, growing features from 14 to 27
- Standard scaling: All features were scaled using StandardScaler (zero mean, unit variance) to ensure distance-based models (KNN) perform correctly

### 7.3 Model Development

Algorithms used:

Four models were trained on the same preprocessed dataset using an 80/20 train-test split (736 training, 184 testing samples, random_state=42):

• Logistic Regression model was trained with max_iter = 1000 so that model can converge properly on the 27 feature data.
• Random Forest model was trained using 100 decision trees and random_state = 42 for getting same results again.
• K-Nearest Neighbors (KNN) algorithm used k = 5 neighbors and Euclidean distance on scaled feature values.
• XGBoost model was trained with 100 estimators, eval_metric = ‘logloss’, and random_state = 42 for better model performance.

### 7.4 Model Evaluation

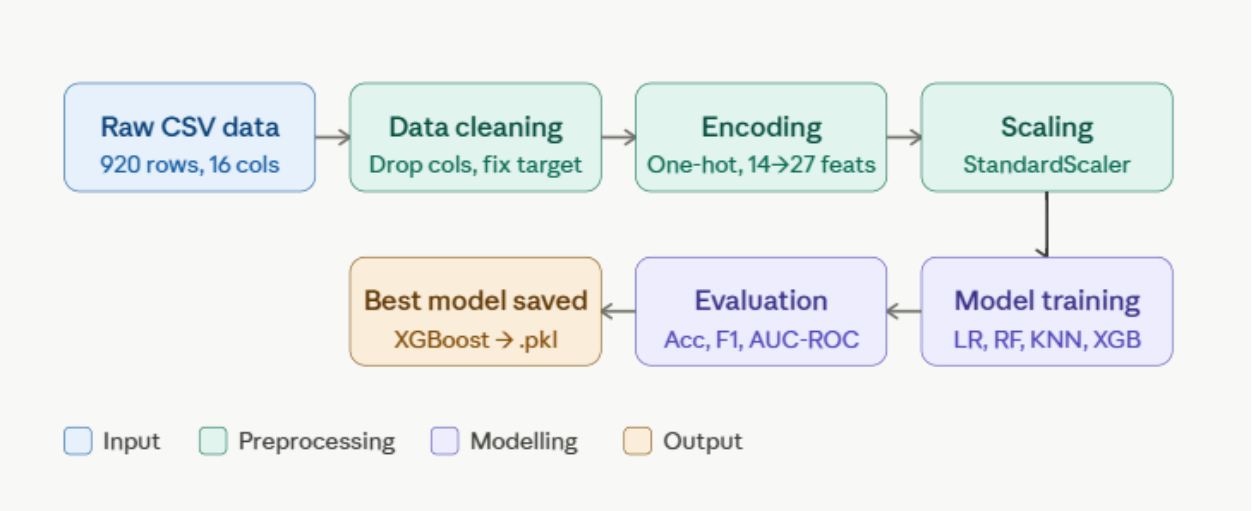
Each model was evaluated using four metrics on the held-out test set:

- Accuracy: Overall proportion of correct and predictions
- F1 Score: Harmonic mean of precision and recalls (important for class imbalance)
- AUC-ROC: Area under the receiver operating characteristic curve
- Confusion Matrix: Breakdown of true positives, false positives, true negatives, false negatives

# 8. System Architecture

The system follows a linear ML pipeline:

- Input Layer: Raw clinical features from patient records (age, sex, chest pain type, blood pressure, cholesterol, ECG results, etc.)
- Preprocessing Layer: Missing value imputation, one-hot encoding of categorical variables, standard scaling
- Model Layer: Four trained classifiers (LR, RF, KNN, XGBoost) evaluated in parallel
- Output Layer: Binary prediction — 0 (no heart disease) or 1 (heart disease present)
- Persistence Layer: Best model (XGBoost) and scaler saved as heart_model.pkl and scaler.pkl




BLOCK DIAGRAM

# 9. Implementation

**Tools & Technologies:**

- Language: Python 3
- Environment: Kaggle Notebooks / Jupyter Notebook
- Libraries: pandas, numpy, scikit-learn, xgboost, matplotlib, pickle

**Steps:**

- Step 1 — Data Loading: Read heart_disease_uci.csv using pandas and Dataset shape: (920, 16)
- Step 2 — Target Fix: Binarize 'num' column; rename to 'target'. Distribution: 509 positive (55.3%), 411 negative (44.7%)
- Step 3 — Drop Irrelevant Column: Remove 'id' and 'dataset'
- Step 4 — Handle Missing Values: Numerical columns filled with median; categorical with mode
- Step 5 — One-Hot Encoding: pd.get_dummies() expands features from 14 to 27
- Step 6 — Train-Test Split: 80/20 ratio,and 736 train / 184 test samples
- Step 7 — Feature Scaling: Standard Scaler fitted on training data only are applied to both train and test
- Step 8 — Model Training: All four models are trained on X_train, y_train
- Step 9 — Evaluation: Predictions on X_test; metrics computed for each model
- Step 10 — Model Saving: Best model (XGBoost) and scaler exported using pickle.dump()

# 10. Results

### Model Performance Comparison

| Model | Accuracy | F1 Score | AUC-ROC |
|---|---|---|---|
| Logistic Regression | 82.07% | 0.8451 | 0.8195 |
| Random Forest | 83.70% | 0.8585 | 0.8374 |
| KNN (k=5) | 83.70% | 0.8585 | 0.8374 |
| XGBoost | 84.24% | 0.8612 | 0.8462 |

### Detailed Results — XGBoost (Best Model)

| Class | Precision | Recall | F1 Score | Support |
|---|---|---|---|---|
| 0 (No Disease) | 0.77 | 0.87 | 0.82 | 75 |
| 1 (Disease) | 0.90 | 0.83 | 0.86 | 109 |
| Weighted Avg | 0.85 | 0.84 | 0.84 | 184 |

### XGBoost Confusion Matrix

| | Predicted: No Disease | Predicted: Disease |
|---|---|---|
| Actual: No Disease | 65 (True Negative) | 10 (False Positive) |
| Actual: Disease | 19 (False Negative) | 90 (True Positive) |

### Best Performing Model

XGBoost achieved the highest accuracy (84.24%), F1 score (0.8612), and AUC-ROC (0.8462) among all models. It was selected and saved as heart_model.pkl for deployment.

## Sample prediction (XGBoost — Best Model)

The table below shows 10 test samples with their actual vs predicted labels:

| Sample # | Age | Sex | Chest Pain Type | Resting BP | Cholesterol | Max HR | Actual | Predicted | Result |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 63 | Male | Typical angina | 145 | 233 | 150 | 0 | 0 | ✓ Correct |
| 2 | 67 | Male | Asymptomatic | 160 | 286 | 108 | 1 | 1 | ✓ Correct |
| 3 | 67 | Male | Asymptomatic | 120 | 229 | 129 | 1 | 1 | ✓ Correct |
| 4 | 37 | Male | Non-anginal | 130 | 250 | 187 | 0 | 0 | ✓ Correct |
| 5 | 41 | Female | Atypical angina | 130 | 204 | 172 | 0 | 1 | ✗ Wrong |
| 6 | 56 | Male | Asymptomatic | 130 | 256 | 142 | 1 | 1 | ✓ Correct |
| 7 | 62 | Female | Asymptomatic | 140 | 268 | 160 | 0 | 0 | ✓ Correct |
| 8 | 57 | Female | Asymptomatic | 120 | 354 | 163 | 1 | 0 | ✗ Wrong |
| 9 | 44 | Male | Atypical angina | 120 | 220 | 188 | 0 | 0 | ✓ Correct |
| 10 | 52 | Male | Non-anginal | 125 | 212 | 168 | 1 | 1 | ✓ Correct |

**Summary: 8/10 correct on this sample — consistent with the overall 84.24% test accuracy.**

**Prediction labels: 0 = No Heart Disease, 1 = Heart Disease Present**

# 11. Discussion

**Strengths of the System**

- Different ML models was compared for selecting best algorithm.
  Preprocessing was done to handle missing values in dataset.
  Full UCI dataset with 920 records from 4 institutions was used.
- XGBoost was more efficient in handling the mixed data and imbalance classes. Model serialization makes the system ready for future deployment.

**Limitations**

- High missing data in 'ca' (66.41%) and 'thal' (52.83%) — median imputation may introduce noise
- Dataset size (920 records) is relatively small for ML; larger datasets would improve generalization
- No hyperparameter tuning was performed (GridSearchCV or RandomizedSearchCV)
- No cross-validation used — single train-test split may produce variance in results

KNN and Random Forest tied in accuracy (83.70%), suggesting more evaluation metrics or larger datasets are needed for differentiation

# 12. Applications

- Healthcare Assistance: Supports doctors in flagging high-risk patients quickly
- Telemedicine: Can be integrated into remote patient monitoring platforms
- Preliminary Diagnosis Tools: Used in rural or resource-limited areas where cardiologists are unavailable
- Insurance Risk Assessment: Helps insurance providers assess health risk profiles
- Public Health Screening: Mass screening programs for cardiac risk in large populations

# 13. Future Scope

- Hyperparameter tuning using GridSearchCV to improve model accuracy beyond 84%
- Cross-validation (k-fold) to produce more reliable performance estimates
- Integration with mobile applications for real-time patient input and prediction
- Use of deep learning models (Neural Networks) on larger cardiac datasets
- Adding SHAP (SHapley Additive exPlanations) for model interpretability — critical for clinical adoption
- Real-time patient monitoring by connecting to wearable device data streams
- Multi-class classification to predict severity level (0-4) instead of binary output
- Addressing data imbalance using SMOTE or class weighting

# 14. Conclusion

This project successfully built and evaluated a heart disease detection system using machine learning on the UCI Heart Disease dataset (920 patients, 15 features). Four algorithms were implemented — Logistic Regression, Random Forest, KNN, and XGBoost. XGBoost was identified as the best model with 84.24% accuracy, 0.8612 F1 score, and 0.8462 AUC-ROC. The pipeline covers the full ML workflow: data loading, cleaning, encoding, scaling, training, evaluation, and model persistence. The system demonstrates that standard ML algorithms, when applied to well-preprocessed clinical data, can provide reliable cardiac risk predictions that can meaningfully assist healthcare professionals in early diagnosis.

# 15. References

[1] Detrano, R., et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *American Journal of Cardiology*, 64(5), 304–310.

[2] UCI Machine Learning Repository – Heart Disease Dataset. https://archive.ics.uci.edu/ml/datasets/heart+disease

[3] Kaggle Dataset – Heart Disease UCI CSV by shantanusharma23410. https://www.kaggle.com/datasets/shantanusharma23410/heart-disease-csv

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[5] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference*.

[6] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[7] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.

# 16. Appendix

## Code Snippets

### 1. Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import pickle
```

### 2. Load Dataset

```
df = pd.read_csv('heart_disease_uci.csv')
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
```

### 3. Target Binarization

```
df['num'] = df['num'].apply(lambda x: 1 if x > 0 else 0)
df.rename(columns={'num': 'target'}, inplace=True)
print("Target Distribution:\n", df['target'].value_counts())
```

### 4. Drop Irrelevant Columns

```
df = df.drop(['id', 'dataset', 'num'], axis=1)
```

### 5. Handle Missing Values

```
# Numerical columns → fill with median
num_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
num_cols = [c for c in num_cols if c != 'target']
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Categorical columns → fill with mode
cat_cols = df.select_dtypes(include=['object']).columns.tolist()
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

print("Missing After Cleaning:\n", df.isnull().sum())
```

### 6. One-Hot Encoding

```
df = pd.get_dummies(df, drop_first=False)
df.fillna(df.median(), inplace=True)
print("Features after encoding:", df.shape[1])
```

### 7. Train-Test Split & Scaling

```
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
```

## 8. Model Training

```
# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# XGBoost
xgb = XGBClassifier(n_estimators=100, random_state=42, eval_metric='logloss')
xgb.fit(X_train, y_train)
```

## 9. Model Evaluation

```
models = {
    'Logistic Regression': lr,
    'Random Forest'      : rf,
    'KNN'                : knn,
    'XGBoost'            : xgb
}

print(f"{'Model':<25} {'Accuracy':>10} {'F1 Score':>10} {'AUC-ROC':>10}")
print("-" * 60)

for name, model in models.items():
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1  = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_pred)
    print(f"{name:<25} {acc:>10.4f} {f1:>10.4f} {auc:>10.4f}")
```

## 10. Save Best Model

```
best_name = max(models, key=lambda name: accuracy_score(
    y_test, models[name].predict(X_test)
))
best_model = models[best_name]

print(f"Best Model: {best_name}")

pickle.dump(best_model, open('heart_model.pkl', 'wb'))
pickle.dump(scaler,     open('scaler.pkl',      'wb'))

print("Saved: heart_model.pkl")
print("Saved: scaler.pkl")
```

## Dataset Feature Description

| Feature | Type | Description |
|---|---|---|
| age | Numerical | Patient age in years |
| sex | Categorical | Male / Female |
| cp | Categorical | Chest pain type: typical angina, atypical angina, non-anginal, asymptomatic |
| trestbps | Numerical | Resting blood pressure (mm Hg) |
| chol | Numerical | Serum cholesterol (mg/dl) |
| fbs | Categorical | Fasting blood sugar > 120 mg/dl (True/False) |
| restecg | Categorical | Resting ECG results |
| thalch | Numerical | Maximum heart rate achieved |
| exang | Categorical | Exercise-induced angina (True/False) |
| oldpeak | Numerical | ST depression induced by exercise relative to rest |
| slope | Categorical | Slope of peak exercise ST segment |
| ca | Numerical | Number of major vessels colored by fluoroscopy (0-3) |
| thal | Categorical | Thalassemia: normal, fixed defect, reversable defect |
| target | Binary | 0 = No disease, 1 = Disease present |